

الفصل الثالث: المصادقة والصلاحيات في منصة «أمان»

ما ستتعلمه في هذا الفصل

- كيف تُبنى سلاسل الـ Middleware في `bootstrap/app.php` وكيف تختلف بين الجماهير الأربعة (`web` ، `user/*` ، `admin/*` ، `guest/*`).
- مفهوم الـ Guard والـ Provider في Laravel، وكيف عُرفت الحُرّاس الثلاثة (`web` ، `admin` ، `user`) في `config/auth.php`.
- كيف يعمل Laravel Sanctum بنظام الـ Bearer Tokens، وكيف تُصدّر الرموز في هذا المشروع، وما هي القدرات (abilities) ومدة الصلاحية الممنوحة لها.
- كل تدفّقات المصادقة عمليًا: دخول الأدمن بكلمة المرور، طلب رمز OTP والتحقّق منه، إعادة تعيين كلمة المرور، تسجيل/دخول المستخدم بالجوّال، تغيير رقم الجوّال، وتسجيل الخروج.
- كيف يُولّد رمز OTP ويُرسَل — بالبريد عبر `SendEmailOTPSERVICE` وبالرسائل القصيرة عبر `SendUserOTPSERVICE` — وأين تُخزّن سجلّاته.
- كيف رُكّبت حزمة `spatie/laravel-permission`، ولماذا تبقى في هذا المشروع طبقة «تزيينية» لا تُنفّذ على الخادم.
- طبقة الاستجابة الموحّدة (`BaseApiController::sendResponse`) وطبقة التحقّق (`CustomFormRequest` + `FailedValidation`).
- أهم الثغرات والعيوب الفعلية في هذه الطبقة، وهي جزء أساسي من قيمة الفصل التعليمية.

ملاحظة منهجية: هذا الفصل يوثّق الكود كما هو موجود فعليًا، لا كما «يجب أن يكون». وفي عدة مواضع سنُشير صريحًا إلى خلل قائم، لأن قراءة الأخطاء في مشروع حقيقي أنفع للمتعلّم من قراءة مثال مثالي.

1. الصورة العامة: أربعة أبواب للـ API

خلّاقًا لمشروع Laravel الافتراضي، لا يوجد في هذا الباب-إند ملف `routes/api.php` إطلاقًا. التوجيه كلّ معرّف يدويًا داخل `bootstrap/app.php` في الدالة `withRouting(then: ...)`، حيث يُحمّل كل ملف مسارات ببائنة (prefix) وسلسلة Middleware خاصة به.

البادئة	ملف المسارات	سلسلة ال Middleware بالترتيب الدقيق	الجمهور
بلا بادئة	routes/web.php	→ web مجموعة → ApiLocalization RequestRateMiddleware	Telescope وفحص الصحة /up
guest/*	routes/guest.php	RequestRateMiddleware → ApiLocalization	الموقع ولوحة التحكم قبل الدخول
admin/*	routes/admin.php	→ auth:sanctum → ApiLocalization RequestRateMiddleware → AdminMiddleware	لوحة التحكم
user/*	routes/user.php	→ auth:sanctum → ApiLocalization RequestRateMiddleware → UserMiddleware	الموقع العام بعد الدخول

الترتيب هنا ليس تفصيلًا شكليًا: `ApiLocalization` يعمل قبل المصادقة، لذا يضبط اللغة لكل طلب حتى للزوّار؛ و `auth:sanctum` يعمل قبل `AdminMiddleware`، لذا حين يصل الطلب إلى `AdminMiddleware` يكون المستخدم قد تمّت مصادقته أصلًا ولم يبقَ إلا تمييز نوعه.

1.1 إعدادات ال Middleware العامة

في `withMiddleware` نجد ثلاثة قرارات مهمّة:

```
$middleware->trustProxies(at: '*');
$middleware->statefulApi();
$middleware->validateCsrfTokens(except: ['*']);
```

- `trustProxies(at: '*')` — الثقة بكل البروكسيات، وهو ما يجعل قراءة عنوان ال IP والبروتوكول تعتمد على ترويسات قد يتحكّم بها العميل.
- `statefulApi()` — يُضيف `EnsureFrontendRequestsAreStateful` في مقدّمة مجموعة `api`، وهو نمط Sanctum المخصّص لجلسات SPA على نفس النطاق.
- `validateCsrfTokens(except: ['*'])` — تعطيل حماية **CSRF** على كل المسارات دون استثناء. الاجتماع بين هذا وبين `statefulApi()` متناقض في المبدأ: الأول يفتح باب مصادقة الجلسات، والثاني ينزع الحماية التي تجعل مصادقة الجلسات آمنة.

كذلك تُسجّل مجموعة اسمها `telescope-auth` تحتوي `AuthenticateWithBasicAuth`، لكنها غير مرفوعة على مجموعة `web` — والتعليق في الكود يوضّح أن ربطها كان يُسبّب خطأ 403 على مسار الشهادة.

1.2 تحويل الاستثناءات إلى مظهر موحّد

في `withExceptions` يُعاد تشكيل استثناءين شائعين حتى لا يخرج HTML للعميل:

الاستثناء	الرسالة	كود HTTP	الشرط
<code>AuthenticationException</code>	<code>Unauthorized</code>	401	للطلبات التي تتوقّع JSON فقط
<code>NotFoundHttpException</code>	<code>EndPoint Is Not Found</code>	404	دائمًا

وهذا يفسّر سبب عدم وجود أي إعادة توجيه (redirect) عند انتهاء الرمز: الواجهات تتلقّى 401 داخل نفس المظهر وتُتصرّف بنفسها.

2. أصناف ال Middleware واحدًا واحدًا

جميع هذه الأصناف ترث `BaseApiController` — وهي حيلة عملية تمكّنها من استدعاء `sendResponse` وإرجاع نفس مظهر الاستجابة الذي تُرجعه المتحكّمات.

2.1 `app/Http/Middleware/API/ApiLocalization.php`

مسؤول عن تحديد اللغة، ويعمل بالخطوات التالية:

1. يقرأ ترويسة `Accept-Language`.
2. يأخذ النص قبل أول فاصلة ، ثم قبل أول شرطة - ، ويحوّله إلى حروف صغيرة. أي أن `ar-SA,en;q=0.9` تصبح `ar`.
3. إن كانت الترويسة غائبة أو اللغة غير موجودة في `config('app.supported_languages')` يرجع إلى `config('app.locale')`.
4. ينفذ `app()->setLocale()`.
5. إن كان هناك مستخدم مصادّق (`auth()->check()`) يكتب العمود `lang` على الموديل في كل طلب. هذه كتابة على قاعدة البيانات مع كل نداء API، وهي تكلفة أداء ينبغي إدراكها.
6. يضبط ترويسة الاستجابة `Content-Language`.

2.2 `UserMiddleware` و `AdminMiddleware`

كلاهما بسيط جدًّا:

```
if (!auth('admin')->check()) {
    return $this->sendResponse(false, [], "You are not admin user", null, 400);
}
```

HTTP كود	الرسالة	الفحص	الصف
400	You are not admin user	auth('admin')->check()	AdminMiddleware
400	You are not user	auth('user')->check()	UserMiddleware

الكود الصحيح دلاليًا هو **403 Forbidden** (مصادق لكن غير مصرّح له)، لكن المشروع يُرجع **400 Bad Request**. هذه ملاحظة مهمة لمن يكتب عميلًا يستهلك الـ API: لا يمكنك التمييز بين «خطأ في المدخلات» و«حُرّاس غير مطابقين» بالاعتماد على كود الحالة وحده.

2.3 app/Http/Middleware/RequestRateMiddleware.php

الاسم يوحي بتحديد المعدّل (rate limiting)، لكن ما يفعله فعليًا هو **التسجيل (logging)**: يُدخل صفاً في جدول **request_rates** مع كل طلب، يحتوي: الطريقة، الرابط، عنوان الـ IP، الـ referer، كامل جسم الطلب بصيغة **JSON**، وكل الترويسات.

النتيجة الأمنية المباشرة: كلمات المرور، ورموز OTP، وترويسة **Authorization: Bearer ...** تُخزن كلها نصًا صريحًا في قاعدة البيانات. أي وصول للقراءة على هذا الجدول يعني اختراقًا كاملاً للحسابات.

3. الحُرّاس والمزوّدون (Guards & Providers)

3.1 مفهوم عام

في Laravel، الـ **guard** يجيب على سؤال «كيف نتعرّف على المستخدم في هذا الطلب؟» (جلسة؟ رمز؟)، و الـ **provider** يجيب على «من أي مصدر ن جلب سجلّه؟» (أي موديل/جدول). فصل السؤالين يسمح بأن يشترك حارسان في نفس الطريقة (Sanctum) وهما يقرآن جدولين مختلفين.

3.2 التطبيق في config/auth.php

الحارس	الطريقة (driver)	المزوّد	الموديل
web	session	users	App\Models\User
admin	sanctum	admins	App\Models\Admin
user	sanctum	users	App\Models\User

- `defaults.guard = sanctum`.
- `defaults.passwords = admins` — وهذه قيمة معطوبة، لأن قسم `passwords` لا يعرّف إلا وسيطًا واحدًا اسمه `users`. أي أن تدقّق إعادة كلمة المرور المدمج في Laravel غير قابل للاستخدام هنا؛ وهو غير مستخدم أصلاً لأن المشروع بنى تدقّقه الخاص بال OTP.

3.3 config/sanctum.php

الإعداد	القيمة	الأثر
guard	<code>['web']</code>	الحارس المستخدم داخليًا لفحص الجلسة قبل اللجوء للرمز
expiration	<code>null</code>	الرموز لا تنتهي صلاحيتها أبدًا
token_prefix	غير مضبوط	لا يمكن لأنظمة كشف التبريات التعرّف على رموز المشروع

آلية العمل مجتمعة: `auth:sanctum` يصادق حامل الرمز، ثم يفرّق الحارسان `admin` و `user` بين نوعي الحاملين اعتمادًا على العمود `tokenable_type` في جدول الرموز.

4. إصدار الرموز والقدرات (Abilities)

كل موضع إصدار في المشروع يستخدم النمط ذاته:

```
$token = $item->createToken("auth_token")->plainTextToken;
```

ثلاث ملاحظات دقيقة على هذا السطر:

1. اسم الرمز دائمًا `"auth_token"` — لا يمكن تمييز جهاز عن آخر من الاسم.
2. لا يُمرّر مصفوفة `abilities`، وقيمتها الافتراضية في Sanctum هي `["*"]`، أي رمز بكامل الصلاحيات دائمًا.
3. لا يُمرّر تاريخ انتهاء، ومع `expiration => null` يصبح الرمز أبدًا.

كلا الموديلين `User` و `Admin` يستخدمان `Laravel\Sanctum\HasApiTokens`.

4.1 مواضع الإصدار

الصف	الدالة	ملاحظة
<code>Admin/AdminAuthController</code>	<code>loginRegisterResendOtp</code>	دخول بكلمة المرور
<code>Admin/AdminAuthController</code>	<code>otpVerify</code>	تحقق OTP
<code>Admin/AdminAuthController</code>	<code>updatePassword</code>	إعادة تعيين كلمة المرور تُسجّل الدخول تلقائيًا
<code>Admin/AdminAuthController</code>	<code>updateEmail</code>	كود ميت: لا مسار مسجّل له، ويتحقق من جدول <code>users</code> بدل <code>admins</code> (خطأ نسخ ولصق)
<code>User/UserAuthController</code>	<code>loginRegister</code>	دخول بلا OTP
<code>User/UserAuthController</code>	<code>otpVerify</code>	تحقق OTP
<code>User/UserAuthController</code>	<code>updateMobile</code>	تغيير الجوال

بعد كل إصدار يُنفَّذ `$item->is_login = true;` — وهي خاصية غير محفوظة وليست عمودًا في الجدول، ولا تظهر في ال `Resources`، فهي عمليًا لا تفعل شيئًا.

هناك أيضًا `app/Models/AccessToken.php` الذي يُخاطب جدول `personal_access_tokens` ويضيف `device_token` إلى `$fillable` (العمود مضاف في المهاجرة `2024_11_01_084556_create_personal_access_tokens_table.php`)، لكن تدفّقات المصادقة لا تستعمل هذا الموديل إطلاقًا.

4.2 إبطال الرموز

الموضع	ما يُبطل
<code>HomeController::logout</code> (<code>POST admin/logout</code>) , <code>POST user/logout</code>	الرمز الحالي فقط
<code>AdminController::destroy</code>	كل رموز الأدمن المستهدف (ويرفض المساس بالمرّف 1)
<code>AdminController::toggleActive</code>	كل رموز الأدمن المستهدف (ويرفض المساس بالمرّف 1)

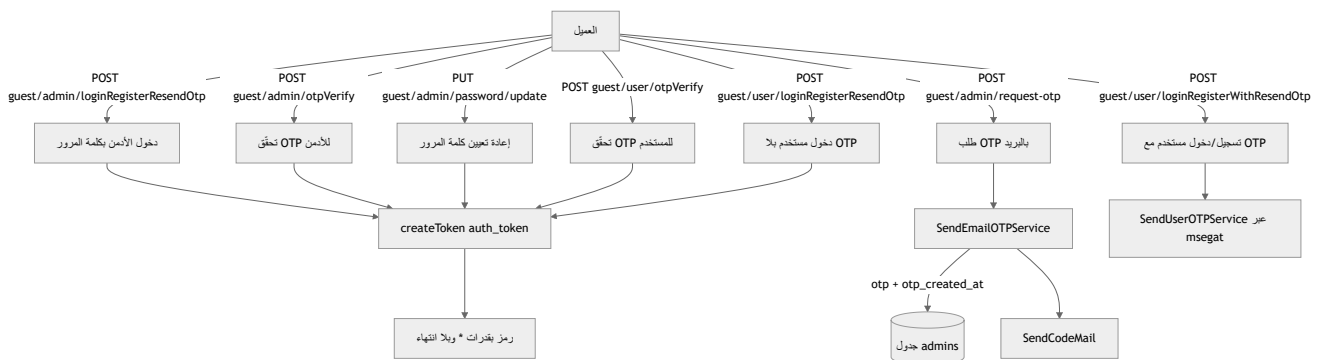
سطر تسجيل الخروج:

```
Auth::user()->tokens()->where('id', Auth::user()->currentAccessToken()->id)->delete();
```

لاحظ أن إعادة تعيين كلمة المرور لا تُبطل الرموز القديمة، فمن سرق رمزًا يبقى داخليًا حتى بعد تغيير الضحية لكلمة مرورها.

5. تدفقات المصادقة بالتفصيل

5.1 نظرة تدفّيقية عامة



5.2 دخول الأدمن بكلمة المرور — POST guest/admin/loginRegisterResendOtp

اسم المسار: `admin.sendotp` — والاسم مُضَيَّل، فهذه الدالة لا ترسل أي OTP.

الدالة: `AdminAuthController::loginRegisterResendOtp`، والتحقق فيها مكتوب داخليًا بـ `Validator`:

الحقل	القاعدة
email	required
password	required

الخطوات: `Admin::where('email', ...)->first()` ثم `Hash::check`، وعند الفشل تُرجع 401 مع `invalidCredentials`. ثم يُحدّث العمود `lang` إلى القيمة الخام لترويسة `Accept-Language` (لا القيمة المنقّاة من `ApiLocalization`)، ويُصدّر الرمز، وتُرجع `{token, item: AdminResource}`.

نقطة مهمّة: هذا هو المسار الوحيد للأدمن الذي لا يستخدم `withTrashed()` ولا يفحص الحساب المحظور/المحذوف منطقيًا. بقية المسارات تفحص `disabledAccount`، أما هذا فلا.

5.3 طلب OTP للأدمن — POST guest/admin/request-otp

الدالة: `AdminAuthController::requestOtp`. القاعدة:

```
.email => required|exists:admins,email,deleted_at,NULL
```

منطق كبح المحاولات (throttle): إذا كان `otp_created_at` مضبوطًا وبيئة التشغيل ليست `testing`، يُقارَن `diffInSeconds` مع `config('constants.otpDelay')`، وعند المخالفة تُرجع 403 مع `{delay_seconds}` ورسالة `.trans('tryAgainAfter')`.

لكن `config/constants.php` يضبط `otpDelay => 0`، أي أن الكبح معطل فعليًا ويمكن طلب رموز بلا حد.

والأخطر: فرع `else` معطوب. الأدمن الذي لم يستلم رمزًا من قبل (`otp_created_at = null`) يتلقّى خطأ 500 مع `emailNotFound`، فلا يستطيع أبدًا طلب أول رمز له.

كما أن الاستجابة تُرجع `ENV` و `ip` والرمز نفسه في كل بيئة غير `production`.

5.4 توليد الرمز وإرساله بالبريد — app/Services/SendEmailOTPService.php

يجري كل العمل في الـ `__construct`:

الحالة	قيمة otp	إرسال البريد
البيئة <code>production</code> أو <code>local</code>	<code>rand(1000, 9999)</code>	نعم
أي بيئة أخرى	القيمة الحرفية <code>'0000'</code>	لا

ثم يُبصَم `otp_created_at`، ويُرسَل البريد عبر `App\Mail\SendCodeMail` (القالب `emails.send-verification-code`، الموضوع `Aman App: OTP`، المُرسِل من `config('mail.from.address')`)، وأخيرًا يُسجَل صف في جدول `otps`:

```
OTP::create(['key' => $email, 'otp' => $otp, 'code' => 200, 'message' => 'Success']);
```

5.5 تحقّق OTP للأدمن — POST guest/admin/otpVerify

الدالة `AdminAuthController::otpVerify(AdminVerifyOtpRequest)`، وتبحث بـ:

```
Admin::withTrashed()->where('email', $request->email)->where('otp', $request->otp)->first();
```


ثم تُحدَّث `lang` وتُصدِر الرمز. ولا تُصدِر الرمز عند النجاح — خلافاً لتدقّق المستخدم — فيبقى قابلاً للاستخدام المتكرّر إلى أن ينتهي زمنياً أو يُستبدّل.

قواعد `app/Http/Requests/AdminVerifyOtpRequest.php` :

الحقل	القاعدة
<code>email</code>	<code>required`</code>
<code>otp</code>	<code>required</code>

وفي `after` → `withValidator` :

الشرط	الحقل	الرسالة
الحساب محذوف منطقياً	<code>email</code>	<code>disabledAccount</code>
لا تطابق للرمز	<code>otp</code>	<code>invalidOtp</code>
<code>Carbon::now()->diffInSeconds(otp_created_at) > 600</code>	<code>otp</code>	<code>otpHasBeenExpired!</code>

مدة الصلاحية 10 دقائق مكتوبة بشكل صريح (hard-coded) في كل صنف طلب على حدة. ولو كان `otp_created_at` فارغاً فالمقارنة تجري ضد «الآن» فتتجح.

5.6 إعادة تعيين كلمة مرور الأدمن — `PUT guest/admin/password/update`

الدالة `AdminAuthController::updatePassword(UpdatePasswordRequest)` تتبع نفس نمط البحث بال OTP، ثم تضبط `password` (يُهشَّم تلقائياً بفضل الـ `cast 'password' => 'hashed'` على `Admin`) مع `lang`، ثم تُصدِر رمزاً وتُسجِّل دخول الأدمن مباشرة. والرمز لا يُصدّر.

قواعد `app/Http/Requests/UpdatePasswordRequest.php` :

الحقل	القاعدة
<code>email</code>	<code>required`</code>
<code>password</code>	<code>required`</code>
<code>otp</code>	<code>required</code>

وتُطبّق نفس فحوص `after()` (حساب معطل / رمز خاطئ / انتهاء 600 ثانية). لا توجد خطوة «كلمة المرور الحالية»، ولا إبطال للرموز القديمة. ولاحظ الحدّ الأعلى `max:12` الذي يمنع كلمات المرور الطويلة القويّة.

5.7 تسجيل/دخول المستخدم مع OTP — POST guest/user/loginRegisterWithResendOtp

اسم المسار `user.registrationResendOtp` ، والدالة

```
.UserAuthController::loginRegisterResendOtp(UserRegisterRequest)
```

يبحث عن `User::withTrashed()->where('mobile', trimMobile(...))` ، وإن لم يجد ينشئ المستخدم تلقائيًا بـ `User::create(['mobile' => ...])` . أي أن التسجيل والدخول نقطة نهاية واحدة. ثم يُطبَّق كبح `otpDelay` نفسه (403 مع `delay_seconds`) ، ويُنادى `new SendUserOTPService($item)` ، وتُرجَع `sms_response_code` و `otpDelay` و `sms_response_message` و `ip` و `ENV` والرمز نفسه إن لم تكن البيئة `production` .

5.8 إرسال الرمز بالرسائل القصيرة — app/Services/SendUserOTPService.php

- `otp = rand(1000, 9999)` دون أي استثناء للبيئات، بخلاف خدمة البريد التي تُثبَّت '0000' في الاختبارات — وهذا يعني أن اختبار تدقّق المستخدم آليًا أصعب.
- يُبصَم `otp_created_at` .
- يُرسل طلب POST إلى `config('msegat.send_url')` بالحقول `apiKey` ، `userSender` ، `numbers` ، `userName` ، `msg` (قالب عربي)، مع `timeout(5)` و `connectTimeout(3)` و `withoutVerifying()` أي تعطيل التحقق من شهادة `**TLS` — ثغرة تسمح بهجوم وسيط.
- عند أي `throwable` يُنادى `saveResponse()` مع القيم الافتراضية `code = 401` و `message = 'API Connection Error'` .
- يكتب `OTP::create([... , 'mobile' => ...])` ، لكن `App\Models\OTP` لا يضمّ `mobile` في `$fillable` ، وجدول `otps` (المهاجرة `2024_07_09_001020_create_otps_table.php`) لا يحتوي إلا `key, otp, code, message` . النتيجة: الإسناد الجماعي يُسقط القيمة صامتا، فتفقد سجلات رسائل SMS هوية المستلم تمامًا.

قواعد `app/Http/Requests/UserRegisterRequest.php` : `mobile => required|min:8|max:20` ، وفي `after()` : الحساب المحذوف منطقياً → `mobile: disabledAccount` ، ويضاف مفتاح خطأ `delay_seconds` عند الوقوع داخل فترة `otpDelay` .

5.9 دخول المستخدم بلا OTP — POST guest/user/loginRegisterResendOtp

هذه أخطر نقطة في المشروع كلّها، ولها سببان مجتمعان:

1. المسار والاسم متبادلان مقارنةً بالفقرة 5.8: المسار `loginRegisterResendOtp` (اسم `user.sendotp`) يُوجّه إلى الدالة `loginRegister` التي لا علاقة لها بال OTP.

2. الدالة `UserAuthController::loginRegister(UserRegisterRequestWithoutOTP)` تجد المستخدم بالجوال أو تنشئه، ثم تُعيد رمزًا كامل الصلاحيات فورًا.

النتيجة: أي رقم جوال يحصل على جلسة مصادقة بلا رمز تحقق وبلا كلمة مرور وبلا أي سرّ. تُرجع الدالة أيضًا `timeout_audio` من `settings('timeout_audio')`. وقواعدها: `mobile => required|min:8|max:20` مع فحص `disabledAccount` فقط.

5.10 تحقّق OTP للمستخدم — `POST guest/user/otpVerify`

الدالة `UserAuthController::otpVerify(UserVerifyOtpRequest)` تطابق على `mobile` + `otp` مع `withTrashed`، ثم:

```
$item->update(['otp' => '', 'mobile_verified_at' => now(), 'lang' => $lang]);
```

هنا خسارة بيانات صامته ثانية: `mobile_verified_at` ليس في `User::$fillable` وليس عمودًا في مهاجرة `0001_01_01_000000_create_users_table.php`، فيُسقط. أي أن المشروع لا يحتفظ بأي أثر لكون الجوال موثّقًا. ثم يُصدّر الرمز مع `timeout_audio`.

قواعد `app/Http/Requests/UserVerifyOtpRequest.php`:

- `otp => required` فقط؛ قاعدة `mobile` معلقة كتعليق في الكود.
- `prepareForValidation` تدمج `mobile => trimMobile($this->mobile)`.
- `after()` : `invalidOtp` / `disabledAccount` / انتهاء 600 ثانية.

ولأن `otp` يُصدّر إلى `''` عند النجاح، فإن البحث `where('otp', '')` كان سيطابق كل الصفوف المصفّرة — لكن قاعدة `required` تمنع إرسال `''` فتسدّ هذه الفجوة عرضًا.

5.11 تغيير رقم الجوال — `PUT user/mobile/update` (يتطلّب مصادقة)

الدالة `UserAuthController::updateMobile` تطابق على `old_mobile` (خامًا، دون `trimMobile`) + `otp`، بالقواعد التالية المكتوبة داخليًا:

الحقل	القاعدة	ملاحظة
<code>old_mobile</code>	<code>required`</code>	<code>sa_mobile</code>
<code>new_mobile</code>	<code>required`</code>	<code>sa_mobile</code>
<code>otp</code>	<code>required</code>	—

وفي `after()` تُضاف فحوص `invalidOtp` و `otpHasBeenExpired!` (600 ثانية). ثم يُصَفَّر الرمز، ويُضَبِّط الجوال الجديد، ويُصدَّر رمز مصادقة جديد.

ملاحظة منطقية جوهرية: لا يُرسَل أي OTP إلى الرقم الجديد، فالتحقُّق يجري على الرقم القديم فقط، وبذلك لا يُثبت المستخدم ملكيته للرقم الذي ينتقل إليه.

5.12 تسجيل الخروج — `POST admin/logout` و `POST user/logout`

`HomeController::logout` يحذف الرمز الحالي فقط ويُرجع الرسالة `(: Logout Success, Bye)`. لا يوجد «تسجيل خروج من كل الأجهزة».

6. الأدوار والصلاحيات

6.1 المفهوم

توفِّر حزمة `spatie/laravel-permission` نموذجًا من ثلاث طبقات: **صلاحية (permission)** وهي إذن ذرّي، ودور **(role)** وهو حزمة صلاحيات، ونموذج **(model)** يمكن ربطه بأي منهما. الفرض (enforcement) يجري عادةً عبر middleware مثل `permission:` و `role:`، أو عبر `$user->can(...)`، أو `authorize()` في المتحكِّم.

6.2 الإعداد الفعلي

- `config/permission.php` : `teams => false` ، `model_morph_key => model_id`
- الجداول من المهاجرة `2025_01_06_130141_create_permission_tables.php` : `roles` ، `permissions` ، `model_has_permissions` ، `model_has_roles` ، `role_has_permissions` — مع إضافة عمودين اختياريين `name_en` و `name_ar` على `permissions`.
- `App\Models\Admin` يستخدم `HasRoles`. أما `App\Models\User` فلا يستخدمها، فالمستخدمون بلا أي نظام صلاحيات.

6.3 لا أدوار على الإطلاق

لا يُنشَأ أي دور **(role)** في هذا المشروع. والعمود `Admin.role_name` هو نصّ حرّ (`nullable|min:5|max:50`) في `AdminStoreRequest`، ويُزرع بالقيمة `'Admin'` ولا علاقة له بأدوار **Spatie**. الصلاحيات تُربط مباشرة بالأدمن:

```
$permissionIds = Permission::whereIn('name', $request->permissions)->pluck('id');
$admin->permissions()->sync($permissionIds);
```

6.4 مجموعة الصلاحيات المزروعة

هي نفسها في `database/seeder/DatabaseSeeder.php` وفي البذرة الاصطناعية

`database/seeder/ProdAdminSeeder.php` ، بـ `guard_name => 'sanctum'` :

المجموعة	الصلاحيات
عام	<code>Website_Management</code> , <code>Overview</code>
المستخدمون	<code>User:Export</code> , <code>User:Delete</code> , <code>User>Edit</code> , <code>User:Add</code>
التوعية	<code>Awareness:Export</code> , <code>Awareness:Delete</code> , <code>Awareness>Edit</code> , <code>Awareness:Add</code> <code>Awareness:View</code>
البرامج	<code>Programs:View</code> , <code>Programs:Export</code> , <code>Programs:Delete</code> , <code>Programs>Edit</code> , <code>Programs:Add</code>

والمهاجرة `2026_07_16_100006_remove_payment_coupon_permissions.php` حذفت كل صلاحيات `Coupon:%` و `Financial:%`.

الأدمن المزروعون: `sarah@aman.com` بكلمة مرور `12345678` (وجوال `966508570275`) في `DatabaseSeeder` ، و `sara@aman.com` بكلمة المرور نفسها في `ProdAdminSeeder` ، وكلاهما مربوط بكل الصلاحيات. وهذه بيانات دخول مكشوفة ومحفوظة في المستودع.

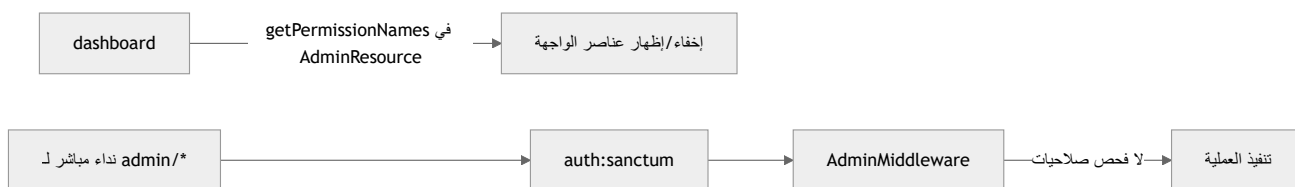
6.5 الفرض: لا شيء على الخادم

البحث في `routes/` و `app/Http/Controllers/` عن `can:` و `permission:` و `role:` و `PermissionMiddleware` و `hasPermissionTo` و `authorize()` يُرجع صفر نتائج.

الخلاصة: أي أدمن مصادق يستطيع نداء كل مسار `admin/*` بغض النظر عن صلاحياته الممنوحة. الصلاحيات في هذا المشروع تُقرأ فقط — تُعرض للوحة التحكم عبر `AdminResource::toArray` باستخدام `$this->getPermissionNames()` لتخفي عناصر الواجهة — وتُكتب في `Admin/AdminController::store` و `::update` . وهذا يعني أن إخفاء زرّ في اللوحة ليس حماية؛ يكفي نداء المسار مباشرة.

مشكلتان إضافيتان في الكتابة:

- الحقل `permissions` غير مُتحقق منه في `AdminStoreRequest` .
- الدالة `update` تنفّذ `sync([])` عند غياب المفتاح `permissions` ، فتسحب صامتًا كل صلاحيات الأدمن.



7. طبقة الاستجابة والتحقق المشتركة

7.1 app/Http/Controllers/BaseApiController.php

الدالة `sendResponse($status, $data, $message, $errors, $code, $request)` تُنتج مظهرًا موحّدًا بالحقول:

الحقل	المعنى
<code>status</code>	نجاح/فشل منطقي
<code>local_language / local</code>	اللغة الحالية
<code>message</code>	رسالة مترجمة
<code>data</code>	الحمولة
<code>guard</code>	الحارس الحالي، من الدالة المساعدة <code>Authed()</code>
<code>errors</code>	خرائط الحقول ورسائلها
<code>response_code</code>	كود HTTP
<code>request_body</code>	كامل الطلب معادًا إلى العميل

حين يكون `status === false` وعدد أوراق `errors` واحدًا أو أقل، تُستبدل بـ `['message' => [$message]]`.
والحقل `request_body` يعيد كلمات المرور ورموز OTP المُرسلة إلى العميل، وهو تسريب واضح في السجلات والوسائط.

توجد كذلك `sendServerError` و `checkValidator($validator)` التي تُرجع استجابة 422 أو `false`.

7.2 الدالة Authed() و app/Services/AuthService.php

الدالة المساعدة Authed() في app/Helpers/general.php:31 تُنشئ AuthService، وفي الـ constructor يُجرب auth('admin')->check() ثم auth('user')->check()، وبناءً على النتيجة تُضبط الخصائص table، guard، id، resource، model، primaryKey:

الحالة	table	resource	guard
أدمن	admins	AdminResource	admin
مستخدم	users	UserResource	user
زائر	null	null	null

وُتُنشأ نسخة جديدة من هذه الخدمة مع كل نداء لـ sendResponse.

7.3 app/Helpers/CustomFormRequest.php

هو الأساس لكل أصناف الطلبات في المشروع:

- authorize() تُرجع true دائماً → لا يوجد أي تصريح على مستوى الطلب في المشروع كله.
- failedValidation ترمي ValidationException حاملةً استجابة app/Services/FailedValidation.php كود 422 بالشكل

```
{status:false, message:<أول خطأ> [+ trans('and') + العدد + trans('moreValidation')], data:null, guard, errors:<[رسائل]>=<[رسائل]>, response_code:422, request_data:<all>}
```

وهذا الشكل بالضبط هو ما تعتمد عليه الدالة handleFormError في الواجهتين لربط الأخطاء بحقول النماذج.

7.4 قاعدة sa_mobile والدالة trimMobile

القاعدة المخصصة sa_mobile مسجلة في app/Providers/AppServiceProvider.php:40 بالتعبير النمطي:

```
/^(970|\+970|0)?5[0-9]{8}$/
```

أي أنها تقبل بادئات فلسطينية (970) رغم أن اسمها ورسالة خطئها

mobileIsNotAValidSaudiArabiaMobileNumber يشيران إلى السعودية. والمفارقة أن تدفقات OTP نفسها (

UserVerifyOtpRequest، UserRegisterRequestWithoutOTP، UserRegisterRequest) لا تستخدم هذه القاعدة بل min:8|max:20 فقط؛ ولا تستعملها إلا updateMobile.

والدالة `trimMobile()` في `app/Helpers/general.php:302` تحذف علامة `+` فقط ولا تؤخذ الصيغة فعليًا — لذلك قد يوجد نفس الرقم بصيغتين مختلفتين.

7.5 الثوابت

في `config/constants.php` يضبط `otpDelay => 0`، أما مدة صلاحية OTP (600 ثانية) فمكثّرة يدويًا في كل صنف طلب بدل أن تكون إعدادًا مركزيًا.

8. ملخص العيوب الأعلى خطورة

#	العيوب	الأثر
1	<code>loginRegister</code> → <code>guest/user/loginRegisterResendOtp</code>	رمز بقدرات <code>*</code> لأي رقم جوال بلا أي سرّ
2	<code>requestOtp</code> وفرع <code>else</code>	الأدمن ذو <code>otp_created_at = null</code> لا يمكنه طلب رمز أبدًا (500)
3	عدم تصفير OTP للأدمن	إعادة استخدام الرمز لعشر دقائق
4	<code>request_body</code> + <code>RequestRateMiddleware</code> + <code>otpDelay = 0</code>	لا كبح للمحاولات، وتخزين وإرجاع كلمات المرور والرموز وال <code>tokens</code>
5	<code>expiration => null</code> + غياب <code>abilities</code>	رموز أبدية كاملة الصلاحية
6	لا فحص صلاحيات على <code>admin/*</code>	طبقة Spatie تزيينية بالكامل
7	<code>mobile_verified_at</code> و <code>mobile</code> غير <code>fillable</code> /غير أعمدة	فقدان بيانات صامت
8	<code>withoutVerifying()</code> في <code>SendUserOTPService</code>	تعطيل التحقق من TLS مع مزوّد SMS
9	في البذور <code>12345678</code>	بيانات دخول مكشوفة في المستودع
10	<code>updateEmail</code> كود ميت يتحقّق من <code>users</code> ؛ <code>updateMobile</code> يتحقّق من <code>brands</code> غير الموجود	أخطاء نسخ ولصق
11	دخول الأدمن بكلمة المرور بلا <code>disabledAccount</code> / <code>withTrashed</code>	تباين في فحوص الحساب المعطل
12	<code>UserMiddleware</code> / <code>AdminMiddleware</code> يُرجعان 400 لا 403	دلالة HTTP خاطئة

أسئلة للمراجعة

- (1) ما الفرق بين `auth:sanctum` و `AdminMiddleware` ، ولماذا نحتاج الاثنين؟ `auth:sanctum` يتحقق من صحة ال Bearer Token ويحدّد صاحبه، أما `AdminMiddleware` فيتحقق أن `auth('admin')->check()` أي أن `tokenable_type` للرمز هو `App\Models\Admin` . بدون الثاني يمكن لرمز مستخدم عادي أن يمرّ إلى مسارات `admin/*` .
- (2) ما القدرات (abilities) ومدّة صلاحية الرموز المُصدّرة في المشروع؟ ولماذا؟ القدرات `["*"]` لأن `createToken("auth_token")` يُنادى بلا وسيط abilities فتُستخدم القيمة الافتراضية. والمدّة غير محدودة لأن `config/sanctum.php` يضبط `expiration => null` ولا يُمرّر تاريخ انتهاء. النتيجة رموز أبدية كاملة الصلاحية.
- (3) لماذا لا يستطيع أدمن جديد طلب رمز OTP؟ لأن `requestOtp` تفحص أولاً `otp_created_at` ؛ وإن كان `null` يذهب التنفيذ إلى فرع `else` الذي يُرجع خطأ 500 برسالة `emailNotFound` بدل إرسال أول رمز.
- (4) ما الفرق الجوهرى بين تحقق OTP للأدمن وللمستخدم؟ تدقّق المستخدم يصقّر العمود بـ `update(['otp' => ''])` بعد النجاح، أما تدقّق الأدمن (`otpVerify` و `updatePassword`) فلا يصقّره، فيبقى الرمز صالحًا للاستخدام مرارًا حتى انتهاء ال 600 ثانية أو استبداله.
- (5) هل إخفاء زرّ في لوحة التحكم بناءً على الصلاحيات يُعدّ حماية؟ لا. الصلاحيات تُقرأ فقط وتُعرض عبر `AdminResource::getPermissionNames()` ، ولا يوجد أي `can:` أو `permission:` أو `authorize()` في المسارات أو المتحكّمات. أي أدمن مصادّق يمكنه نداء أي مسار `admin/*` مباشرة.
- (6) لماذا يفقد جدول `otps` هوية مستليم رسائل SMS؟ لأن `SendUserOTPSERVICE` يمرّر مفتاح `mobile` إلى `OTP::create` ، بينما `App\Models\OTP` لا يضمّه في `$fillable` وجدول `otps` لا يحتوي إلا `key, otp, code, message` . الإسناد الجماعي يُسقط المفتاح صامتًا دون أي خطأ.
- (7) ما الخطر في تسمية المسارات في تدقّق المستخدم؟ المسار `guest/user/loginRegisterResendOtp` (اسم `user.sendotp`) يُوجّه إلى `loginRegister` التي تصدر رمزًا بلا OTP، بينما المسار الذي يرسل OTP فعلاً هو `loginRegisterWithResendOtp` . هذا التبادل يجعل الأول بابًا خلفيًا يمنح جلسة مصادقة لأي رقم جوال.
- (8) ما الوظيفة الحقيقية لـ `RequestRateMiddleware` رغم اسمه؟ لا يحدّد أي معدّل؛ بل يُدخل صفاً في `request_rates` مع كل طلب يحوي الطريقة والرابط وال IP وال referer وكامل جسم الطلب والترويسات — بما يعني تخزين كلمات المرور ورموز OTP وترويسة `Authorization` نصّاً صريحًا.
- (9) لماذا يُعدّ الجمع بين `statefulApi()` و `validateCsrfTokens(except: ['*'])` مشكلة؟ `statefulApi()` يتيح مصادقة قائمة على الجلسة (الكوكيز) للواجهات على نفس النطاق، وهذا النمط يعتمد على حماية CSRF ليكون أمثًا. وتعطيل CSRF على كل المسارات ينزع هذه الحماية.

(10) هل إعادة تعيين كلمة المرور تُبطل الجلسات القديمة؟ وما الأثر؟ لا. `updatePassword` تحدّث كلمة المرور وتُصدر رمزًا جديدًا فقط، بلا حذف للرموز السابقة ولا مطالبة بكلمة المرور الحالية. لذا من استولى على رمز قديم يبقى داخلًا حتى بعد تغيير كلمة المرور.

(11) ما دور `Authed()` و `AuthService` في مخطوف الاستجابة؟ تُحدّد `AuthService` — عبر تجربة `auth('admin')` ثم `auth('user')` — الجدول والموديل والـ Resource والحارس الحالي، ويظهر الحارس في حقل `guard` من كل استجابة. وتُنشأ نسخة جديدة منها مع كل نداء لـ `sendResponse`.

(12) لماذا لا يمكن للمشروع معرفة ما إذا كان جوال المستخدم موثّقًا؟ لأن `otpVerify` تحاول ضبط `mobile_verified_at`، لكنه ليس في `User::$fillable` وليس عمودًا في مهاجرة إنشاء `users`، فتُسقط القيمة صامتا ولا يُخزّن أي أثر للتوثيق.